操作系统原理实验报告

实验名称	授课老师	学生姓名	学生学号
从实模式到保护模式	张青	王健阳	21307261

一、实验要求

1. 突破程序容量不得大于512字节的限制
2. 分别了解使用LBA模式和CHS模式去读取硬盘
3. 实现并调试从实模式到保护模式的过程
4. 尝试在保护模式下编写自定义程序并运行

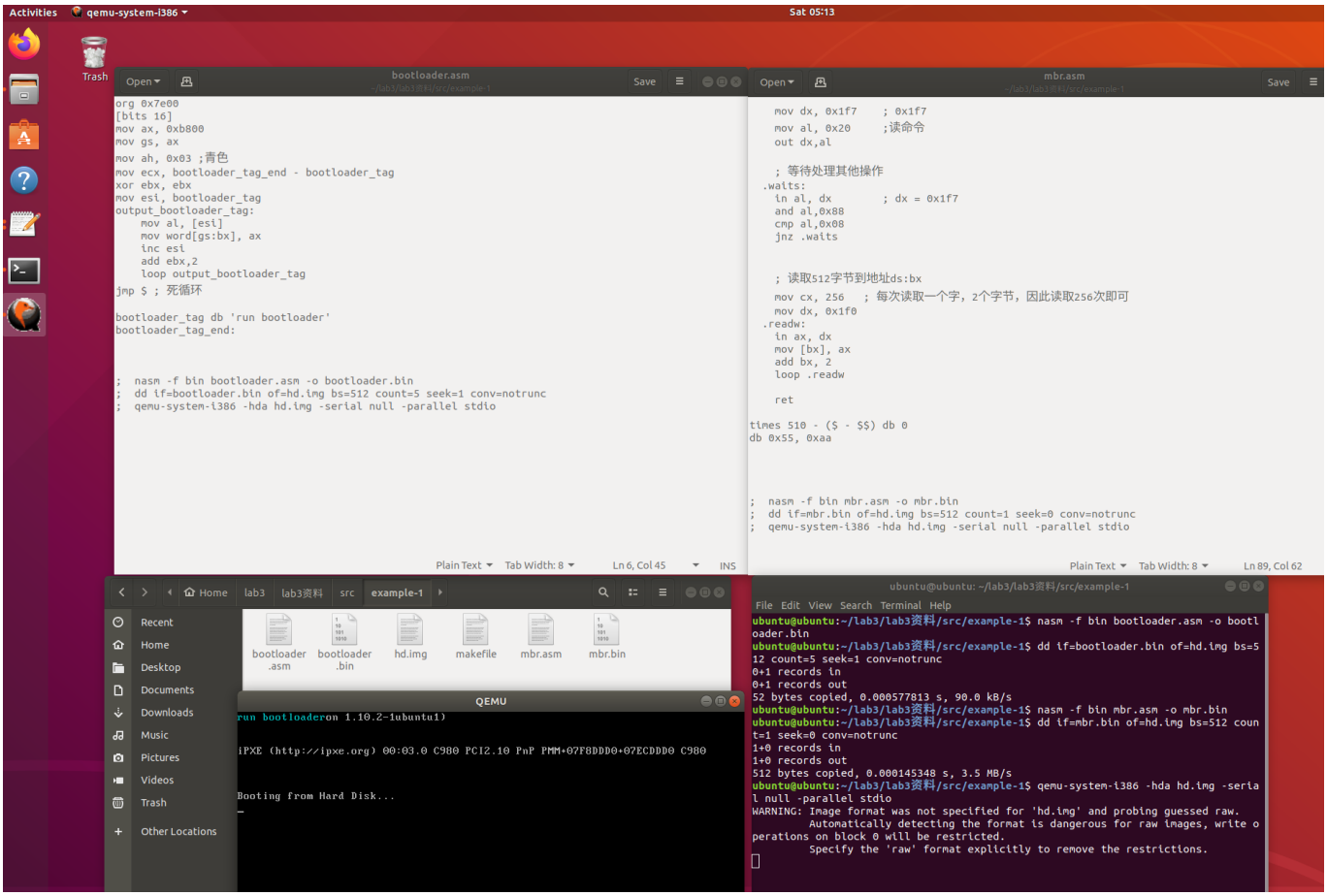
二、实验过程

assignment 1

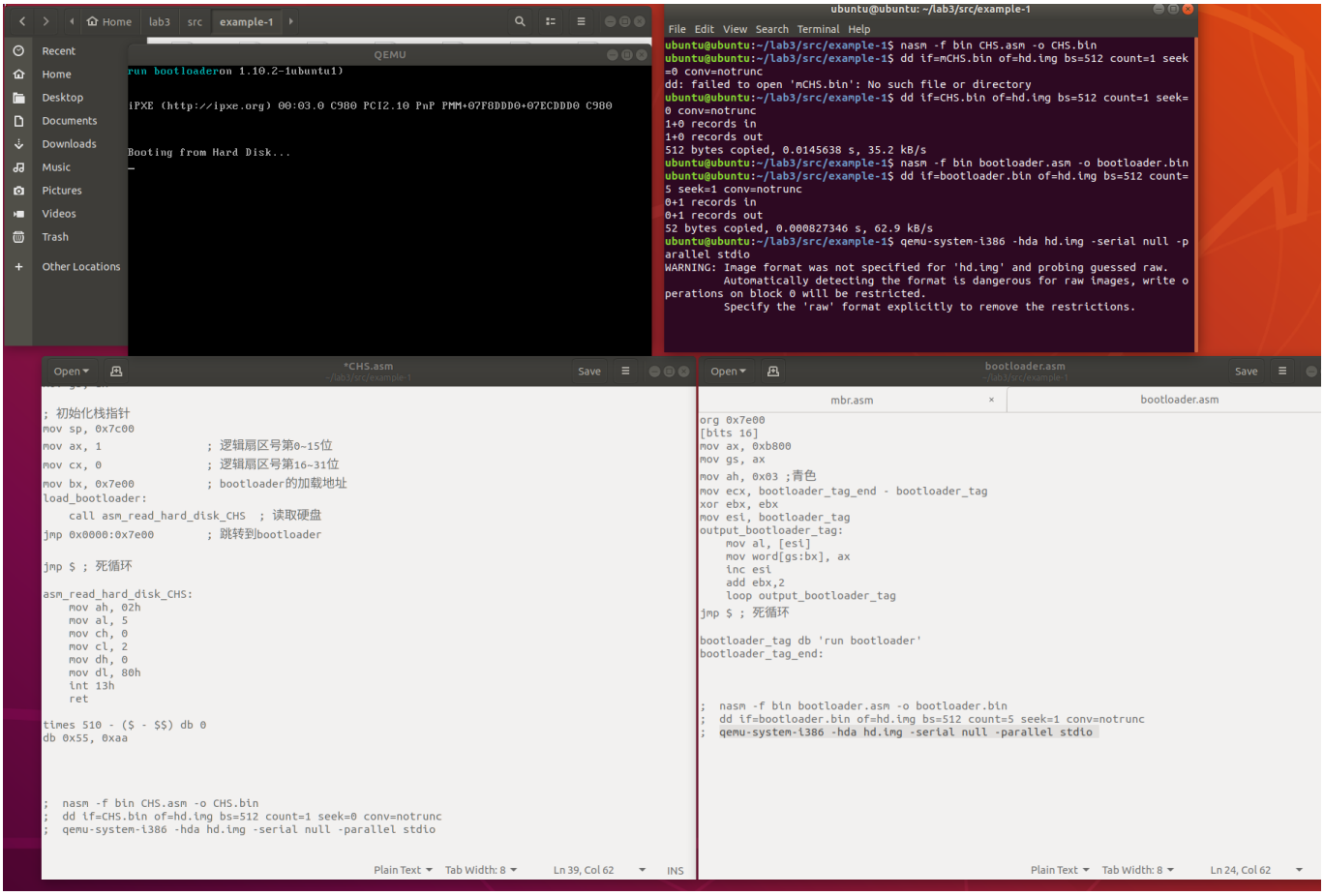
> 1.1 复现example1

编写汇编文件

写入磁盘并运行qemu



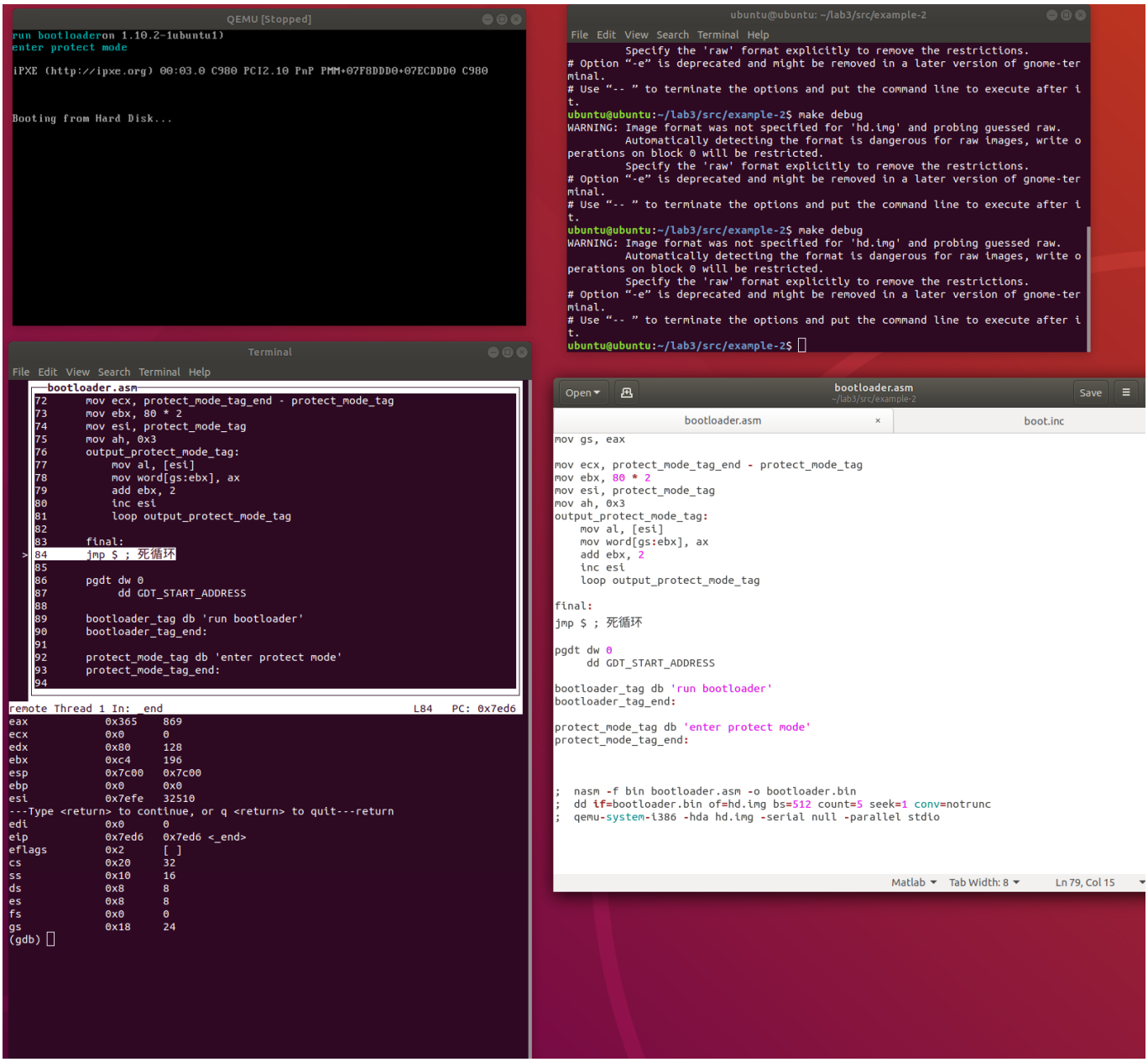
1.2 用CHS模式读取硬盘



assignment 2

复现Example 2，用gdb在进入保护模式的4个步骤上设置断点，并分析这4个步骤

> 用lgdt指令加载GDTR信息
>
> 打开第21根地址线
>
> 开启cr0的保护模式标志位
>
> 远跳转，进入保护模式



先将程序依照每个每个重要步骤进行一个划分，再在每个部分的开头设立标签

用gdb在每个标签处设置断点并运行到此处，依次查看寄存器的值（具体分析在结果处）

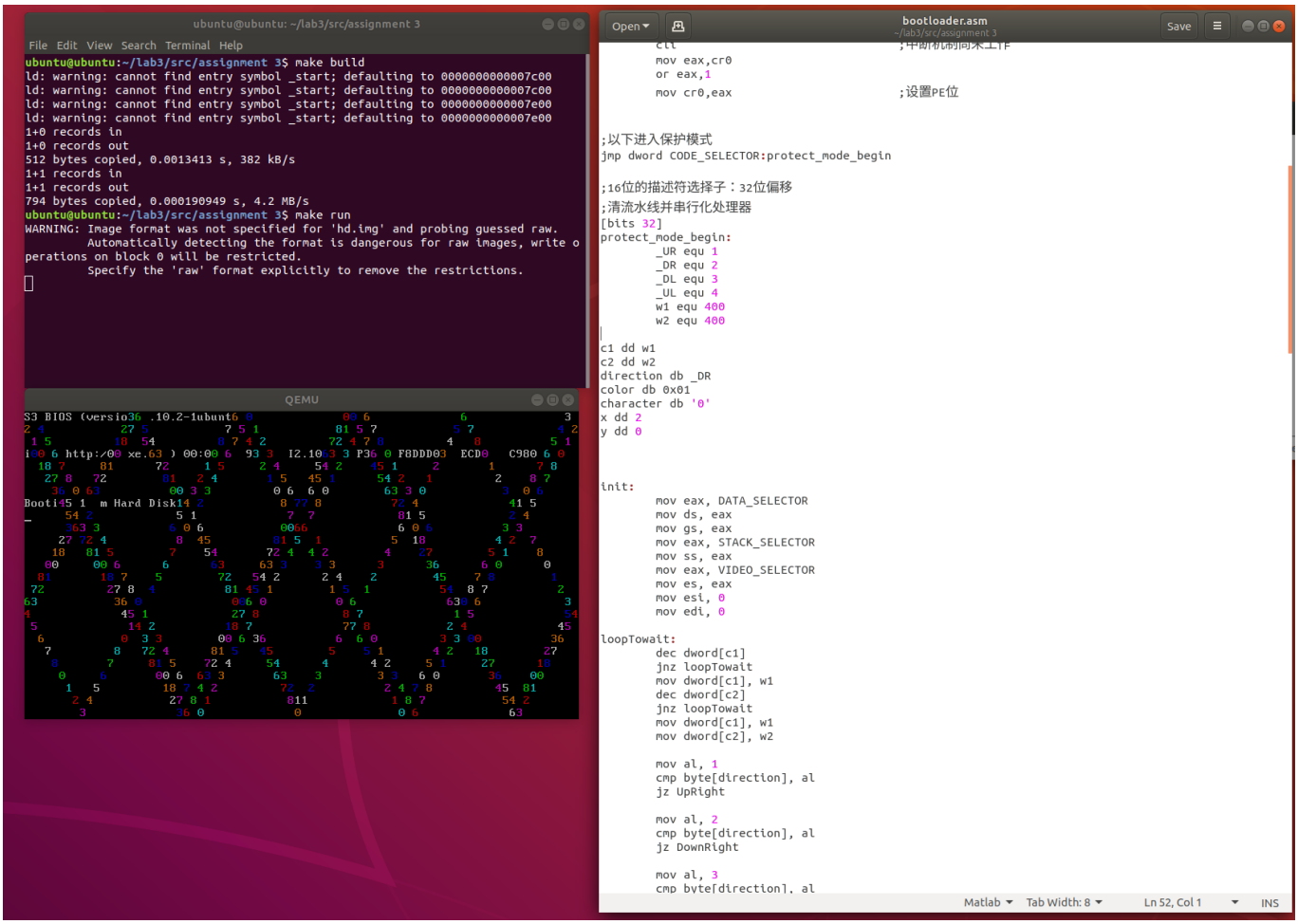
> gdb输入指令（大概）

```
b *0x7c00
c
b *0x7e00
```

```
c
layout src

b GDTR_Init
c
ni * n(运行至该部分结束处，即下一部分的第一条语句处，以下同理)
info registers

b Open_A20
c
ni * n
info registers

b Set_PE
c
ni * n
info registers


c
ctrl c
info registers
```

assignment 3

改造“Lab2-Assignment 4”为32位代码，即在保护模式后执行自定义的汇编程序



改造代码 -> 运行程序

三、关键代码

assignment 1

bootloader.asm（两部分共用代码）

```
org 0x7e00
[bits 16]
mov ax, 0xb800
mov gs, ax
mov ah, 0x03 ;青色
mov ecx, bootloader_tag_end - bootloader_tag
xor ebx, ebx
mov esi, bootloader_tag
output_bootloader_tag:
    mov al, [esi]
    mov word[gs:bx], ax
    inc esi
    add ebx,2
    loop output_bootloader_tag
jmp $ ; 死循环

bootloader_tag db 'run bootloader'
bootloader_tag_end:
```

> 1.1 复现Example 1（此部分为已提供代码）

mbr.asm

```
org 0x7c00
[bits 16]
xor ax, ax ; eax = 0
; 初始化段寄存器，段地址全部设为0
mov ds, ax
mov ss, ax
mov es, ax
mov fs, ax
mov gs, ax

; 初始化栈指针
mov sp, 0x7c00
mov ax, 1                ; 逻辑扇区号第0~15位
mov cx, 0                ; 逻辑扇区号第16~31位
mov bx, 0x7e00           ; bootloader的加载地址
load_bootloader:
    call asm_read_hard_disk  ; 读取硬盘
    inc ax
    cmp ax, 5
```

```
    jle load_bootloader
jmp 0x0000:0x7e00        ; 跳转到bootloader

jmp $ ; 死循环

asm_read_hard_disk:
; 从硬盘读取一个逻辑扇区

; 参数列表
; ax=逻辑扇区号0~15位
; cx=逻辑扇区号16~28位
; ds:bx=读取出的数据放入地址

; 返回值
; bx=bx+512

    mov dx, 0x1f3
    out dx, al    ; LBA地址7~0

    inc dx        ; 0x1f4
    mov al, ah
    out dx, al    ; LBA地址15~8

    mov ax, cx

    inc dx        ; 0x1f5
    out dx, al    ; LBA地址23~16

    inc dx        ; 0x1f6
    mov al, ah
    and al, 0x0f
    or al, 0xe0   ; LBA地址27~24
    out dx, al

    mov dx, 0x1f2
    mov al, 1
    out dx, al   ; 读取1个扇区

    mov dx, 0x1f7    ; 0x1f7
    mov al, 0x20     ;读命令
    out dx,al

    ; 等待处理其他操作
  .waits:
    in al, dx        ; dx = 0x1f7
    and al,0x88
    cmp al,0x08
    jnz .waits

    ; 读取512字节到地址ds:bx
    mov cx, 256   ; 每次读取一个字，2个字节，因此读取256次即可
    mov dx, 0x1f0
  .readw:
    in ax, dx
```

```
    mov [bx], ax
    add bx, 2
    loop .readw

    ret

times 510 - ($ - $$) db 0
db 0x55, 0xaa
```

> 1.2 CHS模式读取硬盘

CHS.asm

```
org 0x7c00
[bits 16]
xor ax, ax ; eax = 0
; 初始化段寄存器，段地址全部设为0
mov ds, ax
mov ss, ax
mov es, ax
mov fs, ax
mov gs, ax

; 初始化栈指针
mov sp, 0x7c00
mov ax, 1                ; 逻辑扇区号第0~15位
mov cx, 0                ; 逻辑扇区号第16~31位
mov bx, 0x7e00           ; bootloader的加载地址
load_bootloader:
    call asm_read_hard_disk_CHS  ; 读取硬盘
jmp 0x0000:0x7e00        ; 跳转到bootloader

jmp $ ; 死循环

asm_read_hard_disk_CHS:  ; 调用Int 13
    mov ah, 02h
    mov al, 5
    mov ch, 0
    mov cl, 2
    mov dh, 0
    mov dl, 80h
    int 13h
    ret

times 510 - ($ - $$) db 0
db 0x55, 0xaa
```

assignment 2

（已提供代码）

mbr.asm

```
%include "boot.inc"

;org 0x7c00
[bits 16]
xor ax, ax ; eax = 0
; 初始化段寄存器，段地址全部设为0
mov ds, ax
mov ss, ax
mov es, ax
mov fs, ax
mov gs, ax

; 初始化栈指针
mov sp, 0x7c00

mov ax, LOADER_START_SECTOR
mov cx, LOADER_SECTOR_COUNT
mov bx, LOADER_START_ADDRESS

load_bootloader:
    push ax
    push bx
    call asm_read_hard_disk  ; 读取硬盘
    add sp, 4
    inc ax
    add bx, 512
    loop load_bootloader

    jmp 0x0000:0x7e00        ; 跳转到bootloader

jmp $ ; 死循环

; asm_read_hard_disk(memory,block)
; 加载逻辑扇区号为block的扇区到内存地址memory

asm_read_hard_disk:
    push bp
    mov bp, sp

    push ax
    push bx
    push cx
    push dx

    mov ax, [bp + 2 * 3] ; 逻辑扇区低16位

    mov dx, 0x1f3
    out dx, al    ; LBA地址7~0

    inc dx        ; 0x1f4
    mov al, ah
```

```
    out dx, al    ; LBA地址15~8

    xor ax, ax
    inc dx        ; 0x1f5
    out dx, al    ; LBA地址23~16 = 0

    inc dx        ; 0x1f6
    mov al, ah
    and al, 0x0f
    or al, 0xe0   ; LBA地址27~24 = 0
    out dx, al

    mov dx, 0x1f2
    mov al, 1
    out dx, al   ; 读取1个扇区

    mov dx, 0x1f7    ; 0x1f7
    mov al, 0x20     ;读命令
    out dx,al

    ; 等待处理其他操作
  .waits:
    in al, dx        ; dx = 0x1f7
    and al,0x88
    cmp al,0x08
    jnz .waits


    ; 读取512字节到地址ds:bx
    mov bx, [bp + 2 * 2]
    mov cx, 256   ; 每次读取一个字，2个字节，因此读取256次即可
    mov dx, 0x1f0
  .readw:
    in ax, dx
    mov [bx], ax
    add bx, 2
    loop .readw

    pop dx
    pop cx
    pop bx
    pop ax
    pop bp

    ret

times 510 - ($ - $$) db 0
db 0x55, 0xaa
```

bootloader.asm

```
%include "boot.inc"

;org 0x7e00
[bits 16]
mov ax, 0xb800
mov gs, ax
mov ah, 0x03 ;青色
mov ecx, bootloader_tag_end - bootloader_tag
xor ebx, ebx
mov esi, bootloader_tag
output_bootloader_tag:
    mov al, [esi]
    mov word[gs:bx], ax
    inc esi
    add ebx,2
    loop output_bootloader_tag


;空描述符
mov dword [GDT_START_ADDRESS+0x00],0x00
mov dword [GDT_START_ADDRESS+0x04],0x00

;创建描述符，这是一个数据段，对应0~4GB的线性地址空间
mov dword [GDT_START_ADDRESS+0x08],0x0000ffff    ; 基地址为0，段界限为0xFFFFF
mov dword [GDT_START_ADDRESS+0x0c],0x00cf9200    ; 粒度为4KB，存储器段描述符

;建立保护模式下的堆栈段描述符
mov dword [GDT_START_ADDRESS+0x10],0x00000000    ; 基地址为0x00000000，界限0x0
mov dword [GDT_START_ADDRESS+0x14],0x00409600    ; 粒度为1个字节

;建立保护模式下的显存描述符
mov dword [GDT_START_ADDRESS+0x18],0x80007fff    ; 基地址为0x000B8000，界限0x07FFF
mov dword [GDT_START_ADDRESS+0x1c],0x0040920b    ; 粒度为字节

;创建保护模式下平坦模式代码段描述符
mov dword [GDT_START_ADDRESS+0x20],0x0000ffff    ; 基地址为0，段界限为0xFFFFF
mov dword [GDT_START_ADDRESS+0x24],0x00cf9800    ; 粒度为4kb，代码段描述符

;初始化描述符表寄存器GDTR
GDTR_Init:
    mov word [pgdt], 39      ;描述符表的界限
    lgdt [pgdt]

Open_A20:
    in al,0x92                          ;南桥芯片内的端口
    or al,0000_0010B
    out 0x92,al                         ;打开A20

Set_PE:
    cli                                 ;中断机制尚未工作
    mov eax,cr0
    or eax,1
    mov cr0,eax                         ;设置PE位
```

```
;55
;以下进入保护模式
jmp dword CODE_SELECTOR:protect_mode_begin

;16位的描述符选择子：32位偏移
;清流水线并串行化处理器
[bits 32]
protect_mode_begin:

mov eax, DATA_SELECTOR                    ;加载数据段(0..4GB)选择子
mov ds, eax
mov es, eax
mov eax, STACK_SELECTOR
mov ss, eax
mov eax, VIDEO_SELECTOR
mov gs, eax
;71
mov ecx, protect_mode_tag_end - protect_mode_tag
mov ebx, 80 * 2
mov esi, protect_mode_tag
mov ah, 0x3
output_protect_mode_tag:
    mov al, [esi]
    mov word[gs:ebx], ax
    add ebx, 2
    inc esi
    loop output_protect_mode_tag

jmp $ ; 死循环

pgdt dw 0
     dd GDT_START_ADDRESS

bootloader_tag db 'run bootloader'
bootloader_tag_end:

protect_mode_tag db 'enter protect mode'
protect_mode_tag_end:
```

assignment 3

assignment3.asm

```
%include "boot.inc"

;org 0x7e00
[bits 16]
;空描述符
mov dword [GDT_START_ADDRESS+0x00],0x00
mov dword [GDT_START_ADDRESS+0x04],0x00
```

```
;创建描述符，这是一个数据段，对应0~4GB的线性地址空间
mov dword [GDT_START_ADDRESS+0x08],0x0000ffff    ; 基地址为0，段界限为0xFFFFF
mov dword [GDT_START_ADDRESS+0x0c],0x00cf9200    ; 粒度为4KB，存储器段描述符

;建立保护模式下的堆栈段描述符
mov dword [GDT_START_ADDRESS+0x10],0x00000000    ; 基地址为0x00000000，界限0x0
mov dword [GDT_START_ADDRESS+0x14],0x00409600    ; 粒度为1个字节

;建立保护模式下的显存描述符
mov dword [GDT_START_ADDRESS+0x18],0x80007fff    ; 基地址为0x000B8000，界限0x07FFF
mov dword [GDT_START_ADDRESS+0x1c],0x0040920b    ; 粒度为字节

;创建保护模式下平坦模式代码段描述符
mov dword [GDT_START_ADDRESS+0x20],0x0000ffff    ; 基地址为0，段界限为0xFFFFF
mov dword [GDT_START_ADDRESS+0x24],0x00cf9800    ; 粒度为4kb，代码段描述符

;初始化描述符表寄存器GDTR
    mov word [pgdt], 39      ;描述符表的界限
    lgdt [pgdt]

    in al,0x92                        ;南桥芯片内的端口
    or al,0000_0010B
    out 0x92,al                       ;打开A20

    cli                               ;中断机制尚未工作
    mov eax,cr0
    or eax,1
    mov cr0,eax                       ;设置PE位


;以下进入保护模式
jmp dword CODE_SELECTOR:protect_mode_begin

;16位的描述符选择子：32位偏移
;清流水线并串行化处理器
[bits 32]
; 自定义32位程序
protect_mode_begin:
    _UR equ 1
    _DR equ 2
    _DL equ 3
    _UL equ 4
    w1 equ 400
    w2 equ 400

c1 dd w1
c2 dd w2
direction db _DR
color db 0x01
character db '0'
x dd 2
y dd 0
```

```
; 寄存器初始化
init:
    mov eax, DATA_SELECTOR
    mov ds, eax
    mov gs, eax
    mov eax, STACK_SELECTOR
    mov ss, eax
    mov eax, VIDEO_SELECTOR
    mov es, eax
    mov esi, 0
    mov edi, 0

loopTowait:
    ; 进行双重循环实现延迟
    dec dword[c1]
    jnz loopTowait
    mov dword[c1], w1
    dec dword[c2]
    jnz loopTowait
    mov dword[c1], w1
    mov dword[c2], w2

    mov al, 1
    cmp byte[direction], al
    jz UpRight

    mov al, 2
    cmp byte[direction], al
    jz DownRight

    mov al, 3
    cmp byte[direction], al
    jz DownLeft

    mov al, 4
    cmp byte[direction], al
    jz UpLeft
    jmp $


; 向右上移动，考虑是否会碰壁
UpRight:
    dec dword[x]
    inc dword[y]

    mov ebx, dword[y]
    mov eax, 80
    sub eax, ebx
    cmp eax, 0
    jz URToUpLeft

    mov ebx, dword[x]
    mov eax, 0
```

```
    sub eax, ebx
    cmp eax, 0
    jg URToDownRight
    jmp show
URToUpLeft:
    mov dword[y], 78
    mov byte[direction], _UL
    jmp show
URToDownRight:
    mov dword[x], 1
    mov byte[direction], _DR
    jmp show


; 向右下移动，考虑是否会碰壁
DownRight:
    inc dword[x]
    inc dword[y]

    mov ebx, dword[y]
    mov eax, 80
    sub eax, ebx
    cmp eax, 0
    jz DRToDownLeft

    mov ebx, dword[x]
    mov eax, 25
    sub eax, ebx
    cmp eax, 0
    jz DRToUpRight
    jmp show
DRToUpRight:
    mov dword[x], 23
    mov byte[direction], _UR
    jmp show
DRToDownLeft:
    mov dword[y], 78
    mov byte[direction], _DL
    jmp show


; 向左下移动，考虑是否会碰壁
DownLeft:
    inc dword[x]
    dec dword[y]

    mov eax, 25
    mov ebx, dword[x]
    sub eax, ebx
    cmp eax, 0
    jz DLToUpLeft

    mov eax, 0
    mov ebx, dword[y]
```

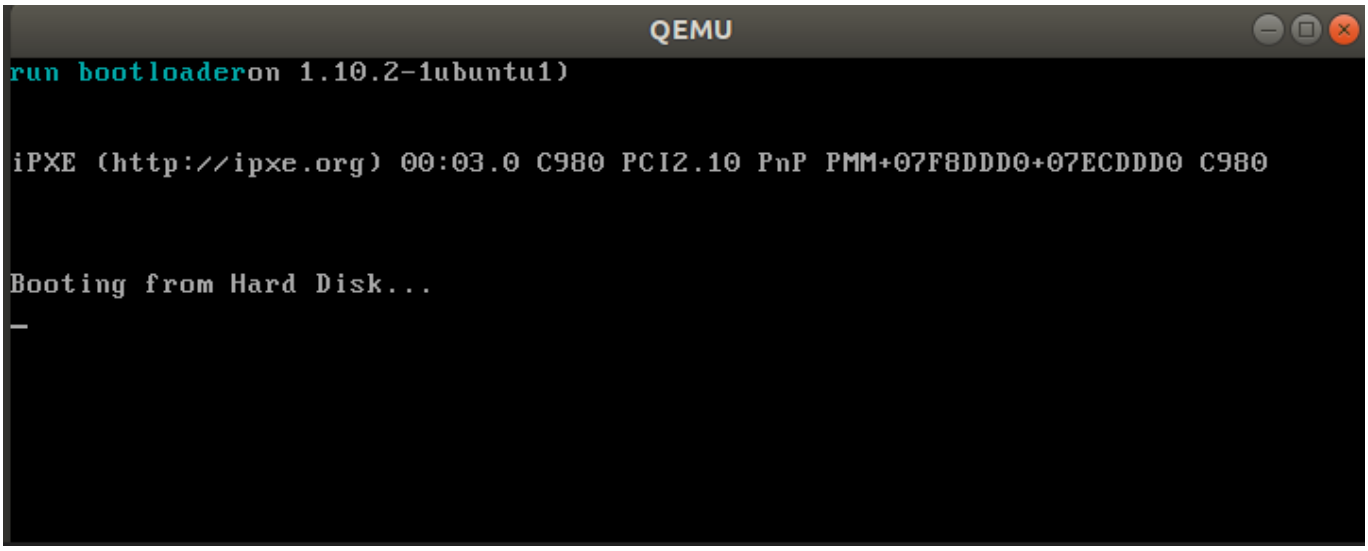
```
    sub eax, ebx
    cmp eax, 0
    jg DLToDownRight
    jmp show
DLToUpLeft:
    mov dword[x], 23
    mov byte[direction], _UL
    jmp show
DLToDownRight:
    mov dword[y], 1
    mov byte[direction], _DR
    jmp show


; 向左上移动，考虑是否会碰壁
UpLeft:
    dec dword[x]
    dec dword[y]

    mov eax, 0
    mov ebx, dword[x]
    sub eax, ebx
    cmp eax, 0
    jg ULToDownLeft

    mov eax, 0
    mov ebx, dword[y]
    sub eax, ebx
    cmp eax, 0
    jg ULToUpRight
    jmp show
ULToDownLeft:
    mov dword[x], 1
    mov byte[direction], _DL
    jmp show
ULToUpRight:
    mov dword[y], 1
    mov byte[direction], _UR
    jmp show


; 显示字符
show:
    xor eax, eax
    mov eax, dword[x]
    mov ebx, 80
    mul ebx
    add eax, dword[y]
    mov ebx, 2
    mul ebx
    mov ebx, eax
    mov al, byte[character]
    mov ah, byte[color]
    mov [es:ebx], eax
```

```
    ; 实现双向射出（关于屏幕中间的y轴对称）
    xor eax, eax
    mov eax, dword[x]
    mov ebx, 80
    mul ebx
    mov ebx, dword[y]
    neg ebx
    add ebx, 79
    add eax, ebx
    mov ebx, 2
    mul ebx
    mov ebx, eax
    mov al, byte[character]
    mov ah, byte[color]
    mov [es:ebx], eax

    ; 改变输出字符的颜色以及种类（控制在0 ~ 9）
    inc byte[color]
    inc byte[character]
    cmp byte[color], 8
    jl color_next
    mov byte[color], 1
color_next:
    cmp byte[character], '9'
    jl character_next
    mov byte[character], '0'
character_next:
    jmp loopTowait
jmp $ ; 死循环

pgdt dw 0
     dd GDT_START_ADDRESS
```

四、实验结果

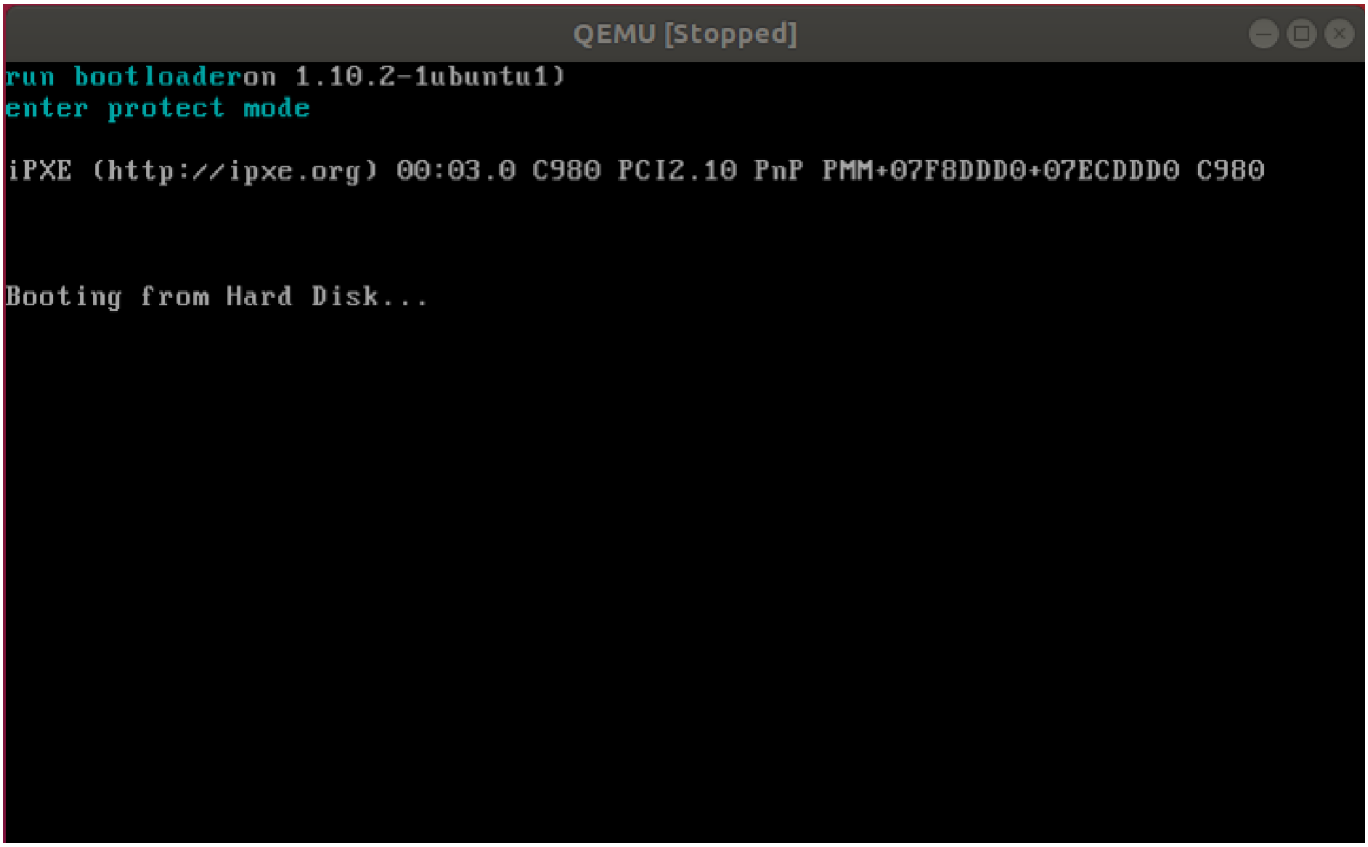
assignment 1

（1.1， 1.2的输出结果一致）



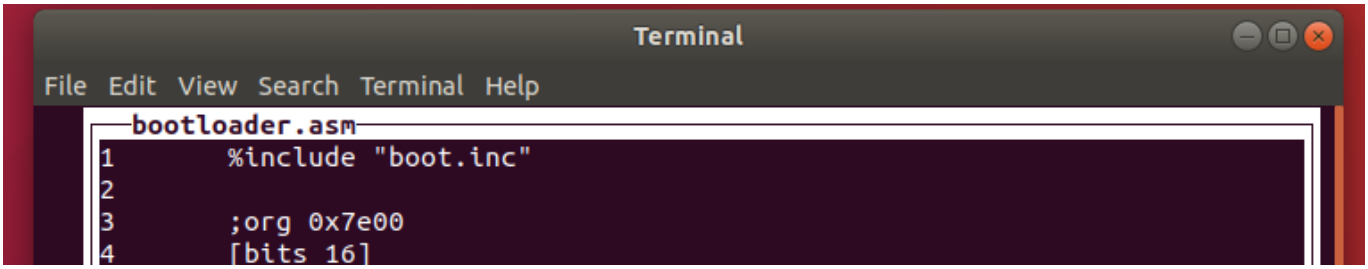
assignment 2

运行结果



调式过程

> 0x7c00, 0x7e00



```
B+>5        mov ax, 0xb800
   6        mov gs, ax
   7        mov ah, 0x03 ;青色
   8        mov ecx, bootloader_tag_end - bootloader_tag
   9        xor ebx, ebx
   10       mov esi, bootloader_tag
   11       output_bootloader_tag:
   12           mov al, [esi]
   13           mov word[gs:bx], ax
   14           inc esi
   15           add ebx,2
   16           loop output_bootloader_tag
   17
   18
   19       ;空描述符
   20       mov dword [GDT_START_ADDRESS+0x00],0x00
   21       mov dword [GDT_START_ADDRESS+0x04],0x00
   22
   23       ;创建描述符，这是一个数据段，对应0~4GB的线性地
   24       mov dword [GDT_START_ADDRESS+0x08],0x0000ffff    ; 基地址为0,
   25       mov dword [GDT_START_ADDRESS+0x0c],0x00cf9200    ; 粒度为4KB,
   26
remote Thread 1 In:                                          L5    PC: 0x7e00
(gdb) info registers
eax            0xaa55   43605
ecx            0x0      0
edx            0x80     128
ebx            0x0      0
esp            0x6f04   0x6f04
ebp            0x0      0x0
esi            0x0      0
edi            0x0      0
eip            0x7c00   0x7c00
eflags         0x202    [ IF ]
cs             0x0      0
ss             0x0      0
ds             0x0      0
es             0x0      0
fs             0x0      0
gs             0x0      0
(gdb) b *0x7e00
Breakpoint 2 at 0x7e00: file bootloader.asm, line 5.
(gdb) c
Continuing.

Breakpoint 2, ?? () at bootloader.asm:5
(gdb) info registers
eax            0x6      6
ecx            0x0      0
edx            0x80     128
ebx            0x8800   34816
esp            0x7c00   0x7c00
ebp            0x0      0x0
esi            0x0      0
edi            0x0      0
eip            0x7e00   0x7e00
eflags         0x286    [ PF SF IF ]
cs             0x0      0
ss             0x0      0
ds             0x0      0
es             0x0      0
fs             0x0      0
gs             0x0      0
```

```
(gdb)
```

上半部分是0x7c00，下半部分是0x7e00

eip的值反映了下一条指令的地址

0x7e00处，ax即将被重新赋值为0xb800，当下的值是仍为（1 + 5 = 6）

（LOADER_START_SECTOR + LOADER_SECTOR_COUNT）

> GDTR_Init

```
Terminal
File  Edit  View  Search  Terminal  Help
bootloader.asm
27      ;建立保护模式下的堆栈段描述符
28      mov dword [GDT_START_ADDRESS+0x10],0x00000000    ; 基地址为0x00
29      mov dword [GDT_START_ADDRESS+0x14],0x00409600    ; 粒度为1个字
30
31      ;建立保护模式下的显存描述符
32      mov dword [GDT_START_ADDRESS+0x18],0x80007fff    ; 基地址为0x00
33      mov dword [GDT_START_ADDRESS+0x1c],0x0040920b    ; 粒度为字节
34
35      ;创建保护模式下平坦模式代码段描述符
36      mov dword [GDT_START_ADDRESS+0x20],0x0000ffff    ; 基地址为0,
37      mov dword [GDT_START_ADDRESS+0x24],0x00cf9800    ; 粒度为4kb,
38
39      ;初始化描述符表寄存器GDTR
40      GDTR_Init:
B+  41              mov word [pgdt], 39      ;描述符表的界限
42              lgdt [pgdt]
43
44      Open_A20:
b+  45              in al,0x92                       ;南桥芯片内的端
46              or al,0000_0010B
47              out 0x92,al                      ;打开A20
48
49      Set_PE:
  >50               cli                              ;中断机制尚未工
51              mov eax,cr0
52              or eax,1
53              mov cr0,eax                      ;设置PE位
54
55
56      ;以下进入保护模式
57      jmp dword CODE_SELECTOR:protect_mode_begin
remote Thread 1 In: Set_PE                          L50   PC: 0x7e8f
(gdb) b GDTR_Init
Breakpoint 2 at 0x7e7e: file bootloader.asm, line 41.
(gdb) c
Continuing.

Breakpoint 2, GDTR_Init () at bootloader.asm:41
(gdb) ni
(gdb) ni
(gdb) info registers
```

```
ecx            0x0      0
edx            0x80     128
ebx            0x1c     28
esp            0x7c00   0x7c00
ebp            0x0      0x0
esi            0x7eec   32492
edi            0x0      0
eip            0x7e89   0x7e89 <Open_A20>
eflags         0x202    [ IF ]
cs             0x0      0
ss             0x0      0
ds             0x0      0
es             0x0      0
fs             0x0      0
gs             0xb800   47104
(gdb) b Open_A20
Breakpoint 3 at 0x7e89: file bootloader.asm, line 45.
(gdb) ni
(gdb) ni
(gdb) ni
(gdb)
```

前面已经输出了run bootloader字符串，故ah的值为0x03，al的值为0x72，gs为显存基地址

GDTR_Init阶段为直接修改主要寄存器的值

> Open_A20

```
Terminal
File  Edit  View  Search  Terminal  Help
bootloader.asm
22
23      ;创建描述符，这是一个数据段，对应0~4GB的线性地址空间
24      mov dword [GDT_START_ADDRESS+0x08],0x0000ffff    ; 基地址为0，段界限为0xFFFFF
25      mov dword [GDT_START_ADDRESS+0x0c],0x00cf9200    ; 粒度为4KB，存储器段描述符
26
27      ;建立保护模式下的堆栈段描述符
28      mov dword [GDT_START_ADDRESS+0x10],0x00000000    ; 基地址为0x00000000，界限0x0
29      mov dword [GDT_START_ADDRESS+0x14],0x00409600    ; 粒度为1个字节
30
31      ;建立保护模式下的显存描述符
32      mov dword [GDT_START_ADDRESS+0x18],0x80007fff    ; 基地址为0x000B8000，界限0x07FFF
33      mov dword [GDT_START_ADDRESS+0x1c],0x0040920b    ; 粒度为字节
34
35      ;创建保护模式下平坦模式代码段描述符
36      mov dword [GDT_START_ADDRESS+0x20],0x0000ffff    ; 基地址为0，段界限为0xFFFFF
37      mov dword [GDT_START_ADDRESS+0x24],0x00cf9800    ; 粒度为4kb，代码段描述符
38
39      ;初始化描述符表寄存器GDTR
40      GDTR_Init:
B+  41              mov word [pgdt], 39      ;描述符表的界限
42              lgdt [pgdt]
43
44      Open_A20:
b+  45              in al,0x92                          ;南桥芯片内的端口
46              or al,0000_0010B
47              out 0x92,al                         ;打开A20
48
49      Set_PE:
>   50              cli                                 ;中断机制尚未工作
51              mov eax,cr0
52              or eax,1
remote Thread 1 In: Set_PE                                          L50   PC: 0x7e8f
ebx            0x1c     28
esp            0x7c00   0x7c00
ebp            0x0      0x0
esi            0x7eec   32492
edi            0x0      0
eip            0x7e89   0x7e89 <Open_A20>
eflags         0x202    [ IF ]
cs             0x0      0
ss             0x0      0
ds             0x0      0
es             0x0      0
fs             0x0      0
gs             0xb800   47104
(gdb) b Open_A20
Breakpoint 3 at 0x7e89: file bootloader.asm, line 45.
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) info registers
eax            0x302    770
ecx            0x0      0
edx            0x80     128
ebx            0x1c     28
esp            0x7c00   0x7c00
ebp            0x0      0x0
esi            0x7eec   32492
edi            0x0      0
eip            0x7e8f   0x7e8f <Set_PE>
eflags         0x202    [ IF ]
cs             0x0      0
ss             0x0      0
ds             0x0      0
es             0x0      0
fs             0x0      0
gs             0xb800   47104
(gdb)
```

此阶段将端口0x92的值读取到al中，与0x2做或运算后写回0x92端口

可以看出eax的值发生了改变

> Set_PE

```
Terminal
```

```
File  Edit  View  Search  Terminal  Help
bootloader.asm
37      mov dword [GDT_START_ADDRESS+0x24],0x00cf9800    ; 粒度为4kb,
38
39      ;初始化描述符表寄存器GDTR
40      GDTR_Init:
B+ 41           mov word [pgdt], 39      ;描述符表的界限
42              lgdt [pgdt]
43
44      Open_A20:
B+ 45           in al,0x92                          ;南桥芯片内的端
46              or al,0000_0010B
47              out 0x92,al                         ;打开A20
48
49      Set_PE:
B+ 50           cli                                 ;中断机制尚未工
51              mov eax,cr0
52              or eax,1
53              mov cr0,eax                         ;设置PE位
54
55
56      ;以下进入保护模式
  > 57  jmp dword CODE_SELECTOR:protect_mode_begin
58
59      ;16位的描述符选择子：32位偏移
60      ;清流水线并串行化处理器
remote Thread 1 In: Set_PE                                  L57   PC: 0x7e9a
esi            0x7eec   32492
edi            0x0      0
eip            0x7e89   0x7e89 <Open_A20>
eflags         0x202    [ IF ]
cs             0x0      0
ss             0x0      0
ds             0x0      0
es             0x0      0
fs             0x0      0
gs             0xb800   47104
(gdb) b Set_PE
Breakpoint 5 at 0x7e8f: file bootloader.asm, line 50.
(gdb) c
Continuing.

Breakpoint 5, Set_PE () at bootloader.asm:50
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) info registers
eax            0x11     17
ecx            0x0      0
edx            0x80     128
ebx            0x1c     28
esp            0x7c00   0x7c00
ebp            0x0      0x0
esi            0x7eec   32492
edi            0x0      0
eip            0x7e9a   0x7e9a <Set_PE+11>
eflags         0x6      [ PF ]
cs             0x0      0
ss             0x0      0
ds             0x0      0
es             0x0      0
fs             0x0      0
```

```
gs             0xb800   47104
(gdb)
```

期间将32位寄存器cr0的值写到eax，并将最低位置为1，eax的值被修改为cr0的值

同时cli指令改变了eflags的值

> 进入保护模式

Terminal

File Edit View Search Terminal Help

bootloader.asm

```asm
    mov dword [GDT_START_ADDRESS+0x24],0x00cf9800    ; 粒度为4kb,

    ;初始化描述符表寄存器GDTR
    GDTR_Init:
            mov word [pgdt], 39      ;描述符表的界限
            lgdt [pgdt]

    Open_A20:
            in al,0x92                          ;南桥芯片内的端
            or al,0000_0010B
            out 0x92,al                         ;打开A20

    Set_PE:
            cli                                 ;中断机制尚未工
            mov eax,cr0
            or eax,1
            mov cr0,eax                         ;设置PE位


    ;以下进入保护模式
    jmp dword CODE_SELECTOR:protect_mode_begin

    ;16位的描述符选择子：32位偏移
    ;清流水线并串行化处理器
    [bits 32]
    protect_mode_begin:

    mov eax, DATA_SELECTOR                      ;加载数据段(0..4GB)
    mov ds, eax
    mov es, eax
    mov eax, STACK_SELECTOR
    mov ss, eax
    mov eax, VIDEO_SELECTOR
    mov gs, eax

    mov ecx, protect_mode_tag_end - protect_mode_tag
    mov ebx, 80 * 2
    mov esi, protect_mode_tag
    mov ah, 0x3
    output_protect_mode_tag:
B+>     mov al, [esi]
        mov word[gs:ebx], ax
        add ebx, 2
        inc esi
        loop output_protect_mode_tag

    _end:
    jmp $ ; 死循环
```

```
86      pgdt dw 0
87           dd GDT_START_ADDRESS
88
89      bootloader_tag db 'run bootloader'
90      bootloader_tag_end:
remote Thread 1 In: output_protect_mode_tag                L77   PC: 0x7eca
(gdb) c
Continuing.

Breakpoint 2, ?? () at bootloader.asm:5
(gdb) b output_protect_mode_tag
Breakpoint 3 at 0x7eca: file bootloader.asm, line 77.
(gdb) c
Continuing.

Breakpoint 3, output_protect_mode_tag () at bootloader.asm:77
(gdb) info registers
eax            0x318    792
ecx            0x12     18
edx            0x80     128
ebx            0xa0     160
esp            0x7c00   0x7c00
ebp            0x0      0x0
esi            0x7eec   32492
edi            0x0      0
eip            0x7eca   0x7eca <output_protect_mode_tag>
eflags         0x6      [ PF ]
cs             0x20     32
ss             0x10     16
ds             0x8      8
es             0x8      8
fs             0x0      0
gs             0x18     24
(gdb)
```

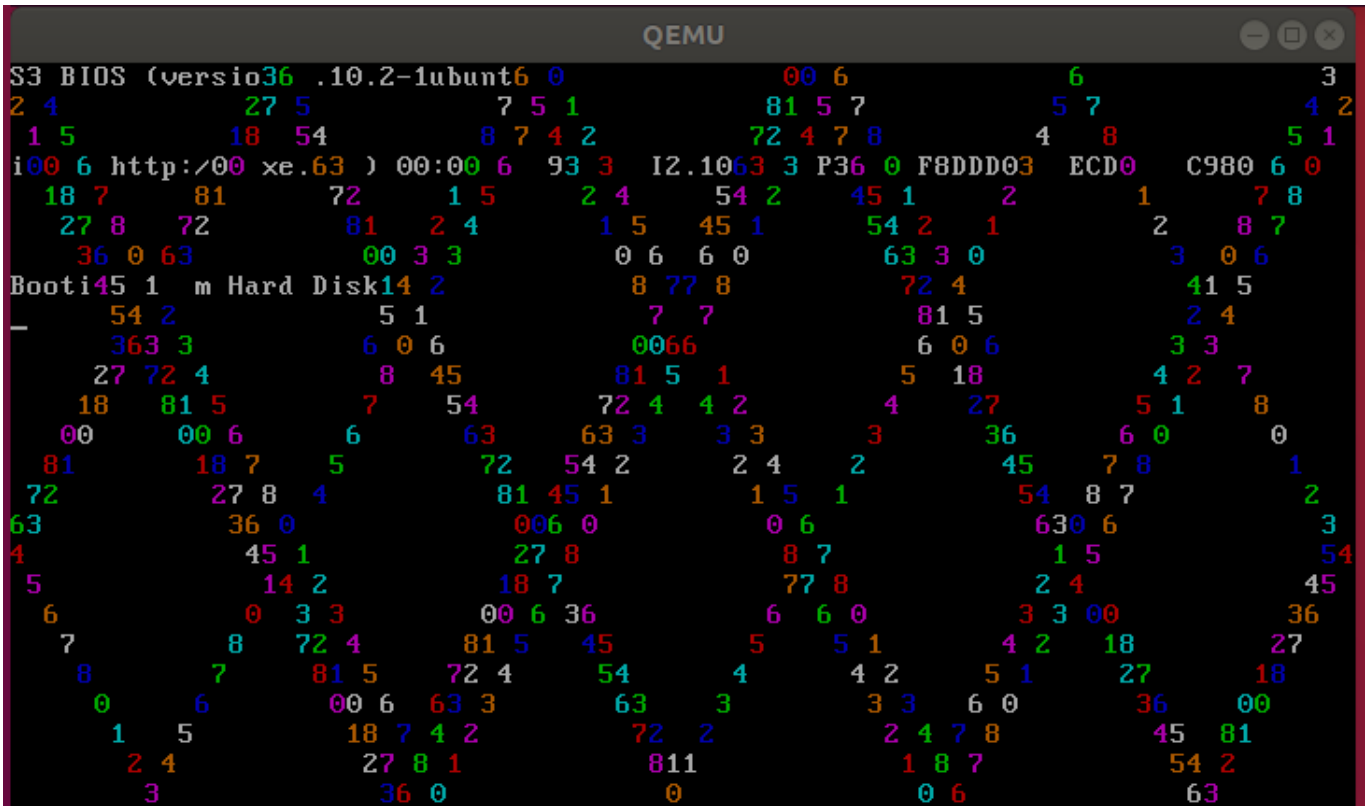
在输出字符串前进行截取，查看到各寄存器被初始化为相应的值

```
Terminal
File  Edit  View  Search  Terminal  Help
bootloader.asm
72      mov ecx, protect_mode_tag_end - protect_mode_tag
73      mov ebx, 80 * 2
74      mov esi, protect_mode_tag
75      mov ah, 0x3
76      output_protect_mode_tag:
77          mov al, [esi]
78          mov word[gs:ebx], ax
79          add ebx, 2
80          inc esi
81          loop output_protect_mode_tag
82
83      final:
> 84      jmp $ ; 死循环
85
86      pgdt dw 0
87           dd GDT_START_ADDRESS
88
89      bootloader_tag db 'run bootloader'
90      bootloader_tag_end:
91
92      protect_mode_tag db 'enter protect mode'
93      protect_mode_tag_end:
94
remote Thread 1 In:  _end                                  L84   PC: 0x7ed6
eax            0x365    869
ecx            0x0      0
edx            0x80     128
ebx            0xc4     196
esp            0x7c00   0x7c00
ebp            0x0      0x0
esi            0x7efe   32510
---Type <return> to continue, or q <return> to quit---return
edi            0x0      0
eip            0x7ed6   0x7ed6 <_end>
eflags         0x2      [ ]
cs             0x20     32
ss             0x10     16
ds             0x8      8
es             0x8      8
fs             0x0      0
gs             0x18     24
(gdb)
```

程序的最末尾再次查看寄存器的值，符合预期

asssignment 3

功能：字符变色，字符转换，双向射出



五、总结

本次实验，让我初步接触到并分别了解了用LBA模式和CHS模式去读取硬盘。知道了去如何使用int 13h中断。在debug的时候也是遇到不少阻碍，首先需要分别删除两个文件里的org语句，make build指令才可以正常运行。其次便是gdbinit里面也有一处地址错误把0x7e00达成了0x7c00，一开始给我造成了很大的困扰（例如添加断点之后程序仍是会继续运行到底，无法进行中途寄存器值的调出）

最后便是借助于assignment 2让我初步了解了makefile文件的使用，在后续的实验中可以不用那么频繁地复制粘贴多条指令了。改进意见：希望以后的实验材料可以稍微完整一点，（例如关于gdb使用的那个文档，看了和没看差别还是很大的）。还有就是关于实验材料的准确性可以再提高一点，尽量减少错误（如果是刻意造成，想提高我们鉴别水平那就算了）